

凸性 min-plus 畳み込み 畳み込み

凸数列の min-plus 畳み込み

[解説動画はこちら](#)です。

1. 概要

2 つの整数列（または実数列） $A = (A_0, A_1, \dots, A_N)$, $B = (B_0, B_1, \dots, B_M)$ に対して、 $C_k = \min_{i+j=k} (A_i + B_j)$ により定義される列 $C = (C_0, \dots, C_{N+M})$ を A, B の **min-plus 畳み込み** といいます。

A, B のうち少なくとも一方に**凸性**がある場合には min-plus 畳み込みを高速に行えることが知られています。本講座ではそのうち最も簡単な場合として、 A, B がどちらも凸である場合の min-plus 畳み込みについて解説します。

この場合の畳み込みは、より具体的な最適化問題と対応させることで非常に直感的に理解することができます。

2. 前提知識

全体を通して必要な知識は特にありませんが、解説の一部では基礎的な dp（動的計画法）が登場します。

3. 凸数列

【定義 1】

整数列 $A = (A_0, A_1, \dots, A_N)$ が任意の i ($0 \leq i \leq N-2$) について不等式 $A_{i+1} - A_i \leq A_{i+2} - A_{i+1}$ を満たすとき、 A は**下に凸** (Convex Below) であるという。逆向きの不等式 $A_{i+1} - A_i \geq A_{i+2} - A_{i+1}$ を満たす場合には、 A は**上に凸** (Convex Above) であるという。

下に凸であるということ、単に凸 (Convex) ということもある。上に凸であることを凹 (Concave) ということもある。

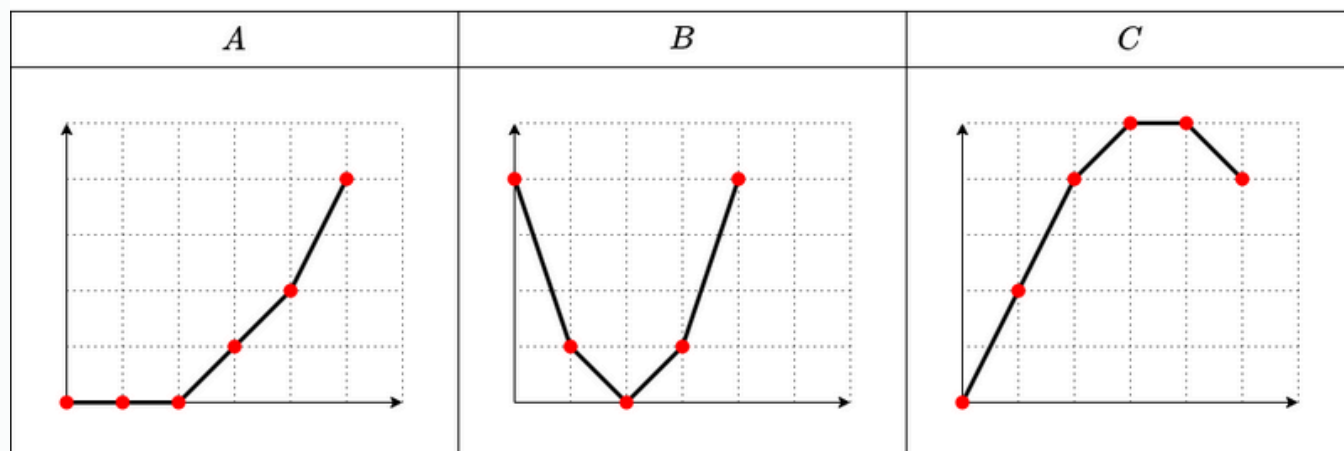
3.1. 具体例と折れ線グラフによる図示

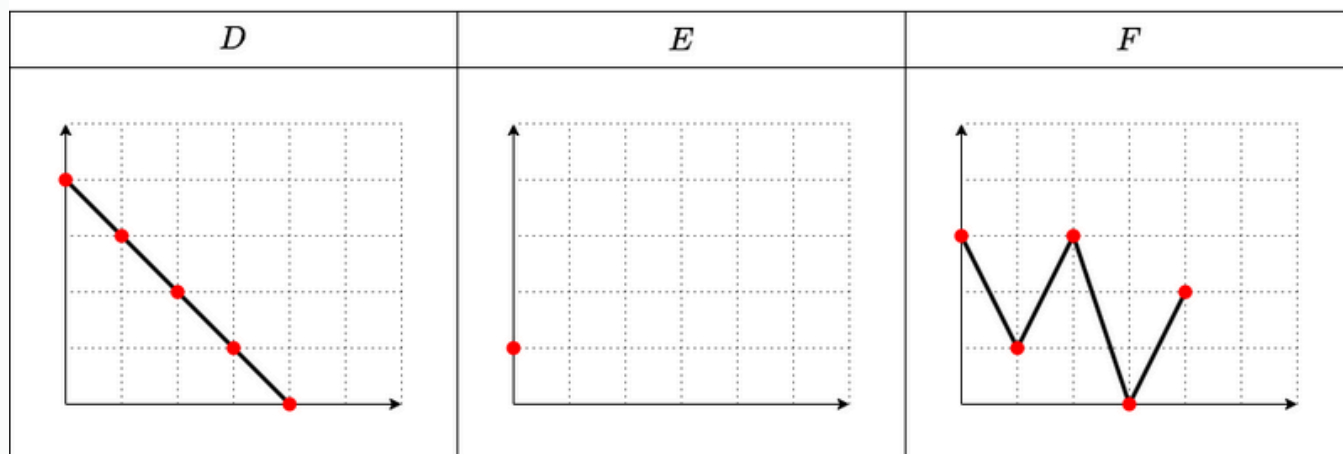
数列 $A = (A_0, A_1, \dots, A_N)$ に対して、座標平面上の点 (i, A_i) を結んでできる**折れ線グラフ**を考えます。

下に凸であるという条件 $A_{i+1} - A_i \leq A_{i+2} - A_{i+1}$ は、折れ線グラフにおいて**傾きが広義単調増加である**という性質と対応しています。

定義式とグラフの形状の両方から、凸数列の具体例を理解してください。

- $A = (0, 0, 0, 1, 2, 4)$ は下に凸である。上に凸ではない。
- $B = (4, 1, 0, 1, 4)$ は下に凸である。上に凸ではない。
- $C = (0, 2, 4, 5, 5, 4)$ は上に凸である。下に凸ではない。
- $D = (4, 3, 2, 1, 0)$ は下に凸でも上に凸でもある。
- $E = (1)$ は下に凸でも上に凸でもある。
- $F = (3, 1, 3, 0, 2)$ は下に凸でも上に凸でもない。





4. 凸数列の min-plus 畳み込み

4.1. 数列の min-plus 畳み込み

【定義 2】

2 つの整数列（または実数列） $A = (A_0, A_1, \dots, A_N)$, $B = (B_0, B_1, \dots, B_M)$ に対して, $C_k = \min_{i+j=k} (A_i + B_j)$ により定義される列 $C = (C_0, \dots, C_{N+M})$ を A, B の **min-plus 畳み込み** (min-plus convolution) という.

例えば, $A = (5, 1, 4, 2, 2)$, $B = (1, 3, 2)$ のとき, これらの min-plus 畳み込みは次のようになります: $C = (6, 2, 4, 3, 3, 4, 4)$.

min-plus 畳み込みは, 定義通りの計算により $O(NM)$ 時間で計算できます. 一般にはそれよりも大きく高速化することは難しい ($\varepsilon > 0$ とするとき, $O((NM)^{1-\varepsilon})$ 時間では解けないだろう) と予想されています (文献 2 を参照).

4.2. 下に凸な数列の min-plus 畳み込み

A, B を下に凸な数列として, これらの min-plus 畳み込みについて, 次を証明することを目指します.

【定理 3】

$A = (A_0, A_1, \dots, A_N)$ と $B = (B_0, B_1, \dots, B_M)$ が下に凸な数列であるとき、これらの min-plus 畳み込みは $O(N + M)$ 時間で計算できる。さらに畳み込みの結果として得られる数列は、下に凸になる。

すべての A_i に定数を加えたり、すべての B_j に定数を加えると、単に畳み込みの結果に定数が加わるだけなので、 $A_0 = 0, B_0 = 0$ の場合に証明すればよいです。以下しばらくこれを仮定します。

A, B の階差数列を a, b とします。つまり、 $a_i = A_{i+1} - A_i, b_j = B_{j+1} - B_j$ によって a_i, b_j を定めます。この場合、 A_i, B_j という値は次のように解釈できます：

- 袋 A に N 個のアイテムが入っており、その価値が $a_0, a_1, \dots, a_{N-1}$ であるとする。 A_i は、これらのアイテムから合計 i 個を選ぶときの合計価値の最小値である。
- 袋 B に M 個のアイテムが入っており、その価値が $b_0, b_1, \dots, b_{M-1}$ であるとする。 B_j は、これらのアイテムから合計 j 個を選ぶときの合計価値の最小値である。

この言い換えに A, B が下に凸である（したがって a, b が広義単調増加である）ことが用いられたことに注意してください。これを踏まえて、 $C_k = \min_{i+j=k} (A_i + B_j)$ は次のように解釈できます。

- $A_i + B_j$ は、袋 A から i 個のアイテム、袋 B から j 個のアイテムを選ぶときの合計価値の最小値である。
- $C_k = \min_{i+j=k} (A_i + B_j)$ は、袋 A, B から合計で k 個のアイテムを選ぶときの合計価値の最小値である。

したがって、 C_k は袋 A, B のアイテムを合わせた $N + M$ 個のアイテムの価値を昇順ソートして、価値の小さいものから k 個の和を求めることで計算できます。

A_0, B_0 を一般にすると、min-plus 畳み込みは次のように計算できることが分かります。

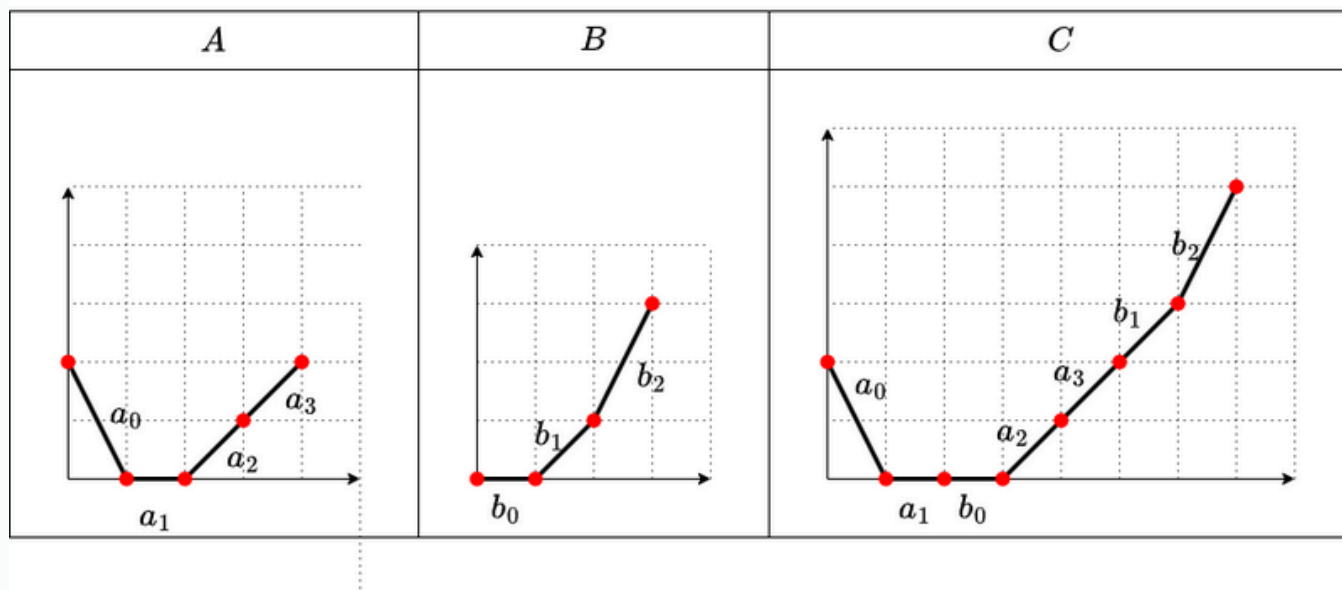
- $a_i = A_{i+1} - A_i, b_j = B_{j+1} - B_j$ とする。
- これらをマージして昇順ソートしたものを $c_0, c_1, \dots, c_{N+M-1}$ とする。
- このとき $C_k = (A_0 + B_0) + (c_0 + c_1 + \dots + c_{k-1})$ である。

「 a_i, b_j をマージして昇順ソートする」という操作が現れますが、 a, b が広義単調増加であることからこの結果は $O(N + M)$ 時間で計算できます。その累積和をとることで、 A, B の min-plus 畳み込みは $O(N + M)$ 時間で計算できます。

C_k の階差数列が広義単調増加であることは計算方法から明らかで、したがって C は下に凸になります。 ■

図解

$A = (2, 0, 0, 1, 2)$ と $B = (0, 0, 1, 3)$ の min-plus 畳み込みは、 $C = (2, 0, 0, 0, 1, 2, 3, 5)$ になります。階差数列が多重集合としてちょうど合併になっていることを確認してみてください。



注意

2 つの数列の max-plus 畳み込みも、min-plus 畳み込みと同様に定義されます。また、上に凸な数列の max-plus 畳み込みについて 【定理 3】 と同様のことが成り立ちます。

4.3. 実装例

Library Checker Min Plus Convolution (Convex and Convex) への提出です。

- <https://judge.yosupo.jp/submission/347758> : 上の証明をそのまま実装したもの。
- <https://judge.yosupo.jp/submission/320371> : 実質的に同じアルゴリズムを、明示的に階差数列をとらずに実装したもの。

5. 階差数列による dp 高速化

競技プログラミングでは min-plus 畳み込みで書ける dp がしばしば登場します。この際、min-plus 畳み込みを線形時間で計算できるということだけでなく、それが階差数列のマージに相当するということを理解しておくことで、dp の高速化ができることがあります。

ここでは次の問題を考えてみます。

【問題 4】

整数列 $A = (A_1, A_2, \dots, A_N)$ が与えられます。 $k = 0, 1, \dots, N$ に対して、 A からちょうど k 個の要素を選んだときの総和としてありうる最小値を求めてください。

もちろんこれは非常に簡単な問題で、昇順ソートして先頭からの累積和をとるだけで答が求まることがすぐに分かると思います（したがって $O(N \log N)$ 時間で解けます）。

一方で、この問題に対して dp による解法も考えられるでしょう。つまり、 $dp[i]$ をちょうど i 個の要素を選んだときの総和としてありうる最小値として、 A_i を追加しながら dp テーブルを更新していくという解法です。例えば次のような実装になります。

```
vector<int> dp = {0};
const int INF = 2000000000; // INF は十分大きな整数
for (int x : A) {
    vector<int> newdp(dp.size() + 1, INF);
    for (int i = 0; i < dp.size(); ++i) {
        newdp[i] = min(newdp[i], dp[i]); // not use x
        newdp[i + 1] = min(newdp[i + 1], dp[i] + x); // use x
    }
    swap(dp, newdp);
}
```

このアルゴリズムの計算量は $\Theta(N^2)$ 時間になってしまいます。この dp を考えてしまった場合、 $O(N \log N)$ 時間で解くのは難しいのでしょうか？

この dp は、min-plus 畳み込みになっています。つまり、数列 B を $B = (0, x)$ により定めると、newdp は dp と B の min-plus 畳み込みです。よって、凸数列の min-plus 畳み込みが階差数列のマージであることを理解していると、この dp 解法からも、

- dp テーブルが常に下に凸であること。

- 階差数列を考えると、dp テーブルの更新は単に x を挿入するという操作であること。

が分かります。よってこの dp から貪欲法による解法と同じ解法が導出できます。

ここで説明した問題例はあまりにも簡単であるために、min-plus 畳み込みの理解が役に立つという実感は得られにくいかもしれません。解説動画では他の問題例も解説しているので、興味があればそちらも確認してみてください。

6. 関連問題

1. https://judge.yosupo.jp/problem/min_plus_convolution_convex_convex
2. https://atcoder.jp/contests/abc407/tasks/abc407_e
3. https://atcoder.jp/contests/joisp2025/tasks/joisp2025_l
4. <https://codeforces.com/contest/2179/problem/E>
5. <https://www.openjudges.com/p/P10641>
6. <https://codeforces.com/contest/2079/problem/B>

▶ 問題へのコメント

7. まとめ

本講座では、下に凸な数列の min-plus 畳み込みのアルゴリズムを、単純な貪欲法により導出しました。仕組みを理解することで、凸数列が与えられたときにその min-plus 畳み込みを計算できるようになるだけでなく、数列の管理方法を工夫してさらに発展的な解法を導出できることもあります。特に dp による解法が min-plus 畳み込みの形になっていることから、階差数列を保持する形式にデータの持ち方を変えることで高速化できるということがあるので、意識してみてください。

本講座の内容は、(実数を定義域とする) 凸関数の畳み込みや、slope trick, 凸図形の Minkowski 和といった事柄の理解にもつながります。また数列の一方のみに凸性がある場合の min-plus 畳み込みについても、今後解説する予定です。

8. 参考文献

1. errorgorn. [Tutorial] Knapsack, Subset Sum and the $(\max,+)$ Convolution. Codeforces Blog. <https://codeforces.com/blog/entry/98663>. (閲覧日: 2026-03-31).
2. Marek Cygan, Marcin Mucha, Karol Woźniak, Michał Włodarczyk. On Problems Equivalent to $(\min,+)$ -Convolution. ACM Transactions on Algorithms. Vol. 15. No. 1. pp. 1–25. 2019. DOI: 10.1145/3293465. <https://doi.org/10.1145/3293465>.

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

各サービス

[AtCoder](#) [🔗](#)

[AtCoderJobs](#) [🔗](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)